

Product Requirements Document: Plumblin

Conversational Bathroom Design with Verified Spatial Compliance AI Lab Case Study —
Track 1: AI Bathroom Designer & Planner | Category: Software | Round: Individual Prototype

1. Executive Summary

Plumblin turns a plain-English request — *"8x10 spa bathroom, family of 4, 6 lakh, touchless"* — into a costed, code-compliant, water-certified bathroom floor plan in under 2 seconds.

The product is named after its verifier, not its generator. A plumb line designs nothing; it only tells you whether what was built is true. That is the architectural thesis: **the language model proposes, deterministic geometry disposes**. An LLM reads intent and makes aesthetic arrangement decisions. A constraint engine decides whether the result is physically legal. Nothing reaches the user that the verifier has not approved.

Positioning: Not a claim that an LLM can do spatial design. The literature and practice both say it cannot do reliable metric geometry. Plumblin's contribution is a **constrained action space plus a verify-repair-fallback loop** that extracts the model's genuine strength (intent, aesthetics, rationale) while making its known weakness (coordinates) structurally impossible to express.

2. Evaluation Criteria Alignment

Criterion	Weight	Plumblin response
Approach & Innovation	45%	Constrained generation — model emits (wall, offset), never x/y. Deterministic verifier as critic. Machine-generated critique fed back for self-repair. Visible reasoning trace.
Technical Execution	25%	26 passing tests. 1.8 ms p50 offline. Runs with no API key. Every LLM stage degrades, none crashes.
User Experience & Feasibility	20%	Single free-text input. SVG plan with clearance overlays. Infeasible inputs explained, not silently failed. Print-ready specification sheet.
Business & Sustainability Impact	10%	WaterSense limits enforced as hard constraints. ~27,900 gal/yr modelled saving for a family of four. Quote-ready SKU output.

3. One-Sentence Definition

We are solving the problem of bathroom fixture selections that fail on budget, clearance or water compliance only after purchase or installation — for homeowners and showroom consultants — by providing a conversational planner whose every output is machine-verified against spatial code and water-efficiency specification before it is shown.

4. Root Cause Analysis

Symptom: Bathroom designs fail at install — fixtures don't fit, doors foul, clearances are illegal, budgets overrun.

- **Why?** Fixture selection and spatial layout are decided separately, by different people, at different times.
- **Why?** Selection happens in a catalogue or showroom; layout happens in a drawing or in someone's head on site.
- **Why?** No tool links a product's dimensions and clearance requirements to the actual room until a designer is paid to draw it.
- **Why?** Doing that link correctly requires enumerating combinations against geometric constraints — tedious manually, and the obvious AI shortcut (ask a model to place fixtures) produces confidently wrong coordinates.
- **Root cause:** The task is a **constraint-satisfaction problem wearing a natural-language interface**. Treating it as purely one or the other fails: pure code can't read "spa-like, family of four"; pure LLM can't guarantee a 21-inch front clearance.

Constraint acknowledgment: Plumblin's premise — that an LLM constrained to `(wall, offset)` and policed by a verifier is more useful than either a bypassed LLM or an unconstrained one — is tested empirically by the repair loop, which logs every rejection. The trace is evidence, not assertion.

5. Users & Stakeholders

User	Need	Decision/Action
Primary: Homeowner	Know what fits, what it costs, what it saves — before committing	Approves or revises a specification
Secondary: Showroom consultant	Produce a compliant, quote-ready bundle in minutes during a walk-in	Issues a quote against real SKUs
Tertiary: Architect / interior designer	Fast feasibility check on a room before detailed drawing	Confirms fixture set fits the shell
Evaluator persona	See genuine AI engineering, not a wrapper	Assesses methodology and rigour

6. Existing Ecosystem & Gap

Existing approach	What it does	Limitation	Our opportunity
Manufacturer catalogue / configurator	Browse and filter products	No room awareness; no clearance or budget validation	Bind product dimensions to the actual room
CAD / BIM (Revit, SketchUp)	Full spatial accuracy	Requires a trained professional; hours of work	Seconds, from one sentence
Generic LLM chat ("design my bathroom")	Reads intent well	Hallucinated coordinates, overlapping fixtures, invented savings figures	Constrain output space; verify every layout; ground every number
Spreadsheet + showroom visit	Real prices	Manual, error-prone, no water modelling	Automated enumeration + WaterSense check

7. Locked Scope Decisions

- LLM action space:** `(wall, offset-along-wall)` per fixture. Raw coordinates are never accepted from the model. Offsets are clipped to the wall run in code.
- Repair budget:** `REPAIR_ATTEMPTS = 2` — three total proposals, then deterministic fallback. Bounded so worst-case latency stays demo-safe.

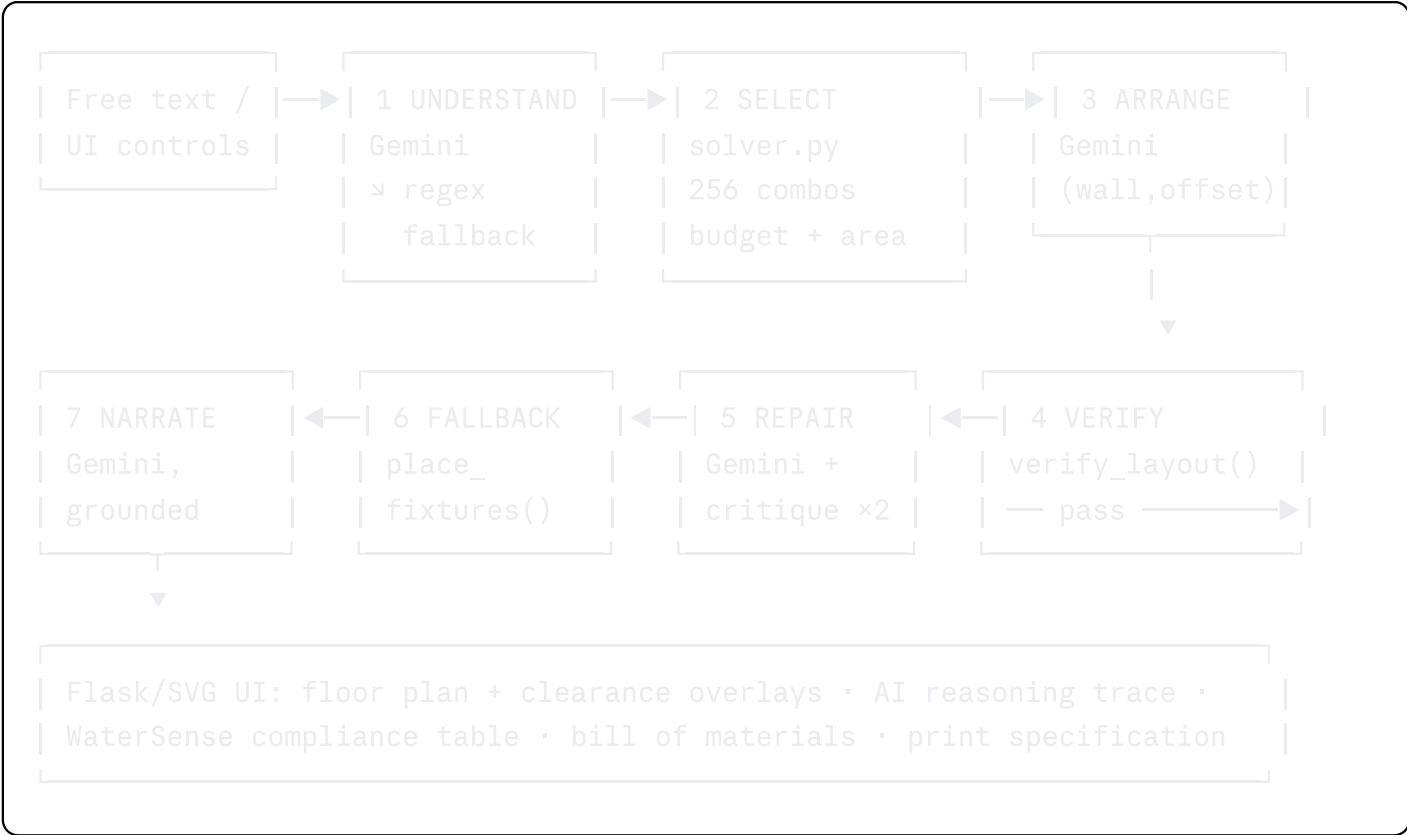
- **Fallback is mandatory, not optional.** If the model fails three times, `place_fixtures()` ships a verified layout. There is no code path that shows an unverified plan.
- **Model:** `gemini-2.5-flash`, overridable via `GEMINI_MODEL`. Chosen for latency, not capability ceiling — the verifier absorbs capability gaps.
- **Catalogue:** 16 products, 4 per category (toilet, shower, vanity, faucet) → **256 enumerable bundles**. A representative sample compiled from public specifications, not a live product feed.
- **Room model:** Single rectangular room, one door at the bottom-left, 30-inch swing square. Multi-door and non-rectangular rooms are out of scope for v1.
- **No database.** Project history is session-scoped. Stated openly rather than faked.
- **Dropped:** a Random Forest "tier recommender" trained on 1,200 synthetically generated labels. It learned a formula the author had written, added heavyweight dependencies, and contributed nothing to any criterion. Removed deliberately, not overlooked.

8. Pipeline Stage Specifications

#	Stage	Owner	Contract
1	Understand	Gemini	Free text → <code>{length_ft, width_ft, budget, theme, household, prioritize_smart, assumed[], reading}</code> . Out-of-range values rejected and replaced from the regex parser.
2	Select	Python	Enumerate 256 bundles → filter by budget and 40% floor-area limit → rank. Cannot return an over-budget or over-footprint bundle.
3	Arrange	Gemini	Emits <code>{wall, offset, reasoning}</code> per fixture. Geometry derived in code via <code>build_placement()</code> .
4	Verify	Python	<code>verify_layout()</code> returns plain-English violations: footprint outside room, clearance outside room, blocks door swing, overlaps, clearance blocked.
5	Repair	Gemini	Violations injected as a critique; model forbidden from repeating the rejected arrangement. Max 2 repairs.
6	Fall back	Python	Depth-first wall-position search, <code>STEP_IN = 6.0 in</code> , <code>MAX_NODES = 15000</code> . Guaranteed-valid or explicitly infeasible.
7	Narrate	Gemini	Receives only verified numbers. Returns <code>{headline, why_this_works, sustainability, tradeoff}</code> . Cannot invent a saving figure because it never sees an unverified one.

Ranking weights (`solver._rank_key`): theme match 100/item · smart features 5/feature (only if requested) · budget utilisation 20× ratio · water penalty 3× (gpf + gpm). Ties break on product ID, so output is deterministic.

9. System Architecture



Design principle: the LLM is never authoritative over anything measurable. It is authoritative over intent and aesthetics only. Every number the user sees originated in `solver.py` or `layout.py`.

Trust boundary: model output crosses into the system only as (a) a validated parameter dict, (b) a `(wall, offset)` pair that code converts to geometry, or (c) prose that references pre-computed values. No model output is ever written directly to a coordinate or a price.

10. Functional Requirements

FR1 — Intent Ingestion Accept free text via `POST /api/design` `{"prompt": "..."} or structured {"params": {...}}. LLM parse with regex fallback; every defaulted field listed in assumed[] and surfaced to the user. Budget parsing handles ₹6,00,000, Rs 6 lakh, 6L, 50k, 1.2 cr, and bare magnitudes with no currency word. Multiplier is required for bare forms so 4 people and 8x10 can never be misread as a budget. Out-of-range LLM values are rejected and replaced, with the rejection recorded in assumed[].`

FR2 — Bundle Selection Enumerate all 256 combinations. Reject any exceeding budget or `FLOOR_USAGE_LIMIT = 0.40`. Rank by the weighted score above. Walk the ranking and return the first bundle with a valid layout.

FR3 — Constrained Layout Generation Model returns `{wall ∈ {top, right, bottom, left}, offset}` per floor fixture. Malformed or incomplete responses trigger a repair round, not a crash. Faucets are mounted on the vanity and consume no floor area.

FR4 — Independent Verification `verify_layout()` re-checks the finished layout from scratch — it does not trust the generator. Checks: footprint inside room, clearance envelope inside room, no door-swing intrusion, no fixture-fixture overlap, no clearance blocked by another fixture.

FR5 — Self-Repair Loop On violation, construct a critique containing the rejected arrangement verbatim and each violation string; instruct the model not to repeat it. Bounded at 2 repairs. Every attempt logged to `data.ai.trace` with attempt number, source, arrangement, reasoning, violations, acceptance, latency.

FR6 — Deterministic Fallback On exhaustion, run constraint search. If no layout exists, return `status: "infeasible"` with a quantified explanation and the nearest feasible alternative and its cost.

FR7 — Water Compliance & Impact Per-product check against EPA WaterSense: toilet ≤ 1.28 gpf, shower ≤ 2.0 gpm, lavatory faucet ≤ 1.5 gpm. Annual saving modelled per fixture class against a legacy 3.5 gpf / 2.5 gpm / 2.2 gpm baseline at 5 flushes, 8 shower-minutes, 2 faucet-minutes per person per day.

FR8 — Grounded Narration Rationale generated from verified metrics only. Returns headline, why-this-works, sustainability, and an honest trade-off.

FR9 — Reasoning Transparency UI renders the full attempt timeline including rejected attempts and their verifier critiques, plus a status pill showing `LLM: live` or `LLM: offline – deterministic engine`. The fallback is disclosed, never hidden.

11. Non-Functional Requirements

Category	Requirement	Measured
Performance	Offline path p50 < 50 ms	1.8 ms p50, 4.5 ms max (7 runs, feasible case)
Performance	LLM path bounded by 3 proposals max	Enforced by <code>REPAIR_ATTEMPTS = 2</code>
Availability	Full functionality with no API key, no network	Verified — <code>/api/health</code> reports <code>gemini: offline</code> , all pages serve
Reliability	No code path shows an unverified layout	Enforced structurally; asserted in tests
Robustness	Malformed LLM JSON, missing fields, wild offsets cannot crash or escape the room	<code>test_geometry_is_never_out_of_bounds</code>
Determinism	Identical input → identical output on the offline path	Tie-break on product ID
Transparency	Every assumed field disclosed; every fallback disclosed	<code>assumed[]</code> , <code>data.ai.layout_source</code>
Honesty	No number shown to the user originates in an LLM	Stage 7 receives only computed values
Testability	Full AI loop testable without an API key	Stubbed-model test

12. Tech Stack

Layer	Technology	Why
Backend	Flask 3	Minimal surface; the intelligence is in the pipeline, not the framework
LLM	Google Gen AI SDK ((google-genai), (gemini-2.5-flash))	Current SDK — the legacy (google.generativeai) package reached end-of-life Nov 2025
Constraint engine	Pure Python ((solver.py), (layout.py))	No dependency; deterministic; unit-testable
Rendering	Vanilla JS + inline SVG with (viewBox) normalisation	Scales to any viewport; prints cleanly
Config	(python-dotenv)	Key never committed
Test	pytest — 26 tests	22 solver/layout + 4 AI-loop

13. API Specification

POST /api/design	{ "prompt": "... " { "params": { ... } } → { status, parsed, data: { bundle, layout, metrics, checks, ai: { trace, layout_source, repairs }, watersense, rationale }, elapsed_ms }
GET /api/health	→ { status, gemini: live offline, model, catalog }
POST /api/projects/save	→ session-scoped project history
GET /planner	→ design workspace
GET /sustainability	→ water impact dashboard

14. MVP Workflow

Free text → LLM intent parse (regex fallback) → 256-combination filter and rank → LLM arrangement proposal → geometric verification → critique and repair (≤2) → deterministic fallback if needed → WaterSense certification → grounded narration → SVG plan + reasoning trace + bill of materials + print specification.

15. Feasibility & Risk Pre-Check

Risk	Mitigation
No API key / network during evaluation	Entire product functions offline; status disclosed, not hidden
LLM returns malformed JSON	Parse failure is a repair round; three failures fall through to the solver
LLM proposes an illegal layout	Exactly what the verifier is for — logged and repaired, visibly
Model deprecation	Overridable via <code>GEMINI_MODEL</code> ; current SDK already migrated
LLM latency spikes mid-demo	Bounded proposal count; deterministic path always available
Prompt injection ("place the toilet in the doorway")	Verifier is downstream of the model and does not read the prompt — an injected instruction cannot bypass a geometric check
Evaluator asks "is this really AI?"	The trace shows the model proposing, failing, being critiqued, and succeeding
Evaluator asks "why not let the AI do coordinates?"	Because it cannot do them reliably — and the repair log is the empirical evidence

16. Scoring Matrix (Self-Assessment)

Factor	Score /10	Note
Innovation	9	Constrained action space + verifier-as-critic is a real architecture, not a wrapper
Technical execution	8	26 tests, sub-2 ms offline, graceful degradation; no persistence layer
Robustness	9	Cannot show an unverified layout; cannot crash on LLM failure
UX	7	Clear and honest; visually plainer than a commercial configurator
Sustainability	8	Real certification thresholds; modelled not metered usage rigour
Business fit	7	Quote-ready output; no live catalogue integration
Honesty	10	Every assumption, fallback and limitation disclosed in-product

Weakest link: catalogue breadth. 16 products is enough to prove the architecture and not enough to be a product. Stated as a limitation rather than disguised.

17. Engineering Practice

Solo build. Practices applied deliberately rather than by team necessity:

Practice	Applied as
Frozen core	<code>solver.py</code> and <code>layout.py</code> treated as immutable during feature work; any diff requires justification
Test gate	26 tests must pass after every change; tests are never edited to make code pass
Structural safety over runtime checks	Constrain the action space so a whole failure class cannot be expressed, rather than validating after the fact
Graceful degradation contract	Every external dependency has a defined offline behaviour before it is added
Honest documentation	README limitations section written before submission, not after being asked
Version control discipline	Baseline commit before any agent-assisted refactor; revert over negotiate

18. 10-Minute Evaluator Test

What problem are we solving? Bathroom designs that fail on budget, clearance or water compliance after purchase — because product selection and spatial layout are decided by different people at different times, and no tool binds them.

Who is the user? Homeowners and showroom consultants who need a compliant, costed, quote-ready specification in minutes, not a CAD session.

What's wrong with current approaches? Catalogues have no room awareness. CAD needs a professional and hours. Generic LLMs produce overlapping fixtures and invented savings figures with total confidence.

What makes ours different? The model never emits a coordinate. It chooses a wall and an offset; code derives the geometry; an independent verifier approves or rejects; rejections become a critique the model must answer. If it fails three times, a constraint search ships a valid plan anyway. The user sees the whole argument.

Why not just let the AI do it? Because it can't, and we can prove it — the repair log shows the model failing and recovering on real requests.

Why not just use code? Because code can't read "spa-like for a family of four." Intent and aesthetics are genuinely the model's job.

What does this save the business? A non-compliant layout is caught at specification time by the verifier rather than on site by an installer. Every rejected attempt in the trace is a rework cycle that never happened.

19. Open Items

- Catalogue is a 16-product representative sample from public specifications; live product-feed integration is unbuilt.
- Water savings are modelled from published flow ratings and assumed usage patterns, not metered data. The usage assumptions are stated in `solver.py` and changeable in one place.
- Single door position and rectangular rooms only.
- No persistence, authentication or multi-user support; session-scoped only.
- Regional plumbing code profiles (IS 1172 vs IBC/ADA) are designed for — clearances are already data, not logic — but only one profile is populated.
- Repair-rate and intent-accuracy metrics are logged per request but not yet aggregated across a benchmark set.